Операционные системы

Лекция 6

Диспетчеризация и планирование

План лекции

- Понятия диспетчеризации и планирования
- Задача планирования
- Алгоритмы планирования
- Особенности многопроцессорного планирования
- Функции для управления планированием

Диспетчеризация и планирование

К этому моменту мы рассмотрели две концепции для понимания как выполняются программы, а именно: процессы и потоки

Также стало понятно, что именно данные понятия лежат в основе многозадачности

В многозадачных системах в основной памяти одновременно содержится код нескольких процессов. В работе каждого процесса периоды использования процессора чередуются с ожиданием некоторых событий, таких как завершение выполнения операций ввода-вывода

Процессор (или процессоры) занят выполнением одного процесса, в то время как остальные находятся в состоянии ожидания

Диспетчеризация и планирование

Ну и конечно же складывается ситуация когда в состоянии готовности одновременно находятся два или более процесса или потока. А значит все они претендуют на получение процессорного времени

Если доступен только один центральный процессор (ну или таких процессоров меньше готовых процессов), необходимо выбрать, какой из этих процессов будет выполняться следующим

Та часть операционной системы, на которую возложен этот выбор, называется **планировщиком**, а алгоритм, который ею используется, называется **алгоритмом планирования**

Диспетчеризация и планирование

Многие однотипные вопросы, применяемые к планированию процессов, могут применяться и к планированию потоков, хотя существуют и некоторые различия

Когда ядро управляет потоками, планирование обычно касается каждого из потоков, практически не различая, какому именно процессу они принадлежат (или все же делая небольшие различия)

Во многом всё зависит от того, какие потоки поддерживаются системой – уровня ядра или уровня пользователя

Далее мы будем говорить о планировании процессов (но по сути под этим мы понимаем некоторое задание которое может быть представлено и как процесс, и как поток)

Диспетчеризация и планирование

Немного истории, о том как мы пришли к тому, что имеем:

В первых поколениях операционных систем алгоритм планирования был достаточно прост: требовалось всего лишь запустить следующее задание (всё просто, пакеты заданий шли последовательно друг за другом)

С появлением многозадачных систем алгоритм планирования усложнился, поскольку в этом случае обычно фигурировали сразу несколько пользователей, ожидавших обслуживания. Поскольку на таких машинах процессорное время является дефицитным ресурсом, хороший планировщик может существенно повлиять на ощущаемую производительность машины и удовлетворенность пользователя. Поэтому на изобретение искусного и эффективного алгоритма планирования было потрачено немало усилий

Диспетчеризация и планирование

Немного истории, о том как мы пришли к тому, что имеем:

С появлением персональных компьютеров ситуация изменилась в двух направлениях. Во-первых, основная часть времени отводилась лишь одному активному процессу – тому, в котором работал пользователь в данный момент времени. Во-вторых, с годами компьютеры стали работать настолько быстрее, что центральный процессор практически перестал быть дефицитным ресурсом. Поэтому на простых персональных компьютерах планирование не играет особой роли

Но в серверах за процессорное время соревнуются уже несколько процессов, поэтому планирование снова приобретает значение. Еще одно важное направление – мобильные устройства, в которых планировщики стараются еще и оптимизировать потребление электроэнергии

Диспетчеризация и планирование

Планировщик наряду с выбором «правильного» процесса должен заботиться также об эффективной загрузке центрального процессора, поскольку переключение процессов является весьма дорогостоящим занятием

Сначала должно произойти переключение из пользовательского режима в режим ядра, затем сохранено состояние текущего процесса, включая сохранение его регистров в таблице процессов для их последующей повторной загрузки

На некоторых системах должна быть сохранена также карта памяти (например, признаки обращения к страницам памяти)

После этого запускается алгоритм планирования для выбора следующего процесса

Диспетчеризация и планирование

Затем в соответствии с картой памяти нового процесса должен быть перезагружен блок управления памятью. И наконец, новый процесс должен быть запущен

Вдобавок ко всему перечисленному, переключение процессов обесценивает весь кэш памяти, заставляя его дважды динамически перезагружаться из оперативной памяти (после входа в ядро и после выхода из него)

В итоге слишком частое переключение может поглотить существенную долю процессорного времени, что наводит на мысль: этого нужно избегать

Все действия по непосредственному переключению с одного процесса на другой есть ни что иное как **диспетчеризация**

Диспетчеризация и планирование

В общем виде процесс диспетчеризации можно представить следующим образом:

- Формируется контекст прерываемого процесса
- С помощью контекста вновь активизируемого процесса восстанавливается необходимое состояние вычислительной системы, в частности, загружаются необходимые значения во все регистры процессора
- Поскольку в регистр-счетчик команд из контекста заносится адрес очередной подлежащей выполнению команды активизируемого процесса, то процессор переходит к выполнению кода нового процесса точно с того места, где оно было прервано

Виды планирования

Как мы уже выяснили ключом к эффективной многозадачности является **планирование**

Обычно используются четыре типа планирования:

Долгосрочное планирование	Решение о добавлении процесса в пул выполняемых процессов
Среднесрочное планирование	Решение о добавлении процесса к числу процессов, полностью или частично размещенных в основной памяти
Краткосрочное планирование	Решение о том, какой из доступных процессов будет выполняться процессором
Планирование ввода-вывода	Решение о том, какой из запросов процессов на операции ввода-вывода будет обработан доступным устройством ввода-вывода

Виды планирования

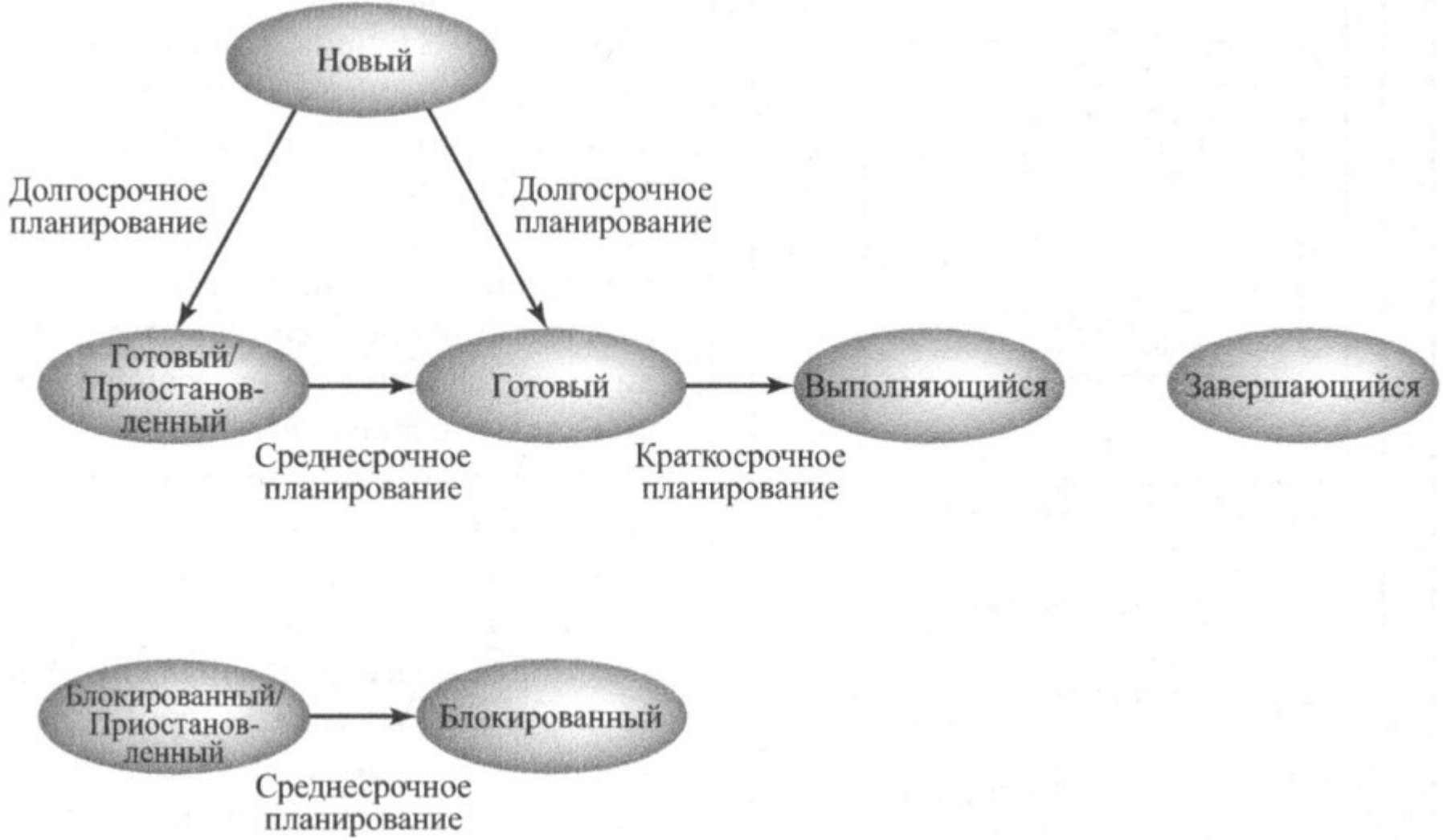
Во многих системах планирование разбивается на три отдельные функции – долгосрочного, среднесрочного и краткосрочного планирования. Их названия соответствуют временным масштабам выполнения этих функций

Долгосрочное планирование осуществляется при создании нового процесса и представляет собой решение о добавлении нового процесса к множеству активных в настоящий момент процессов

Среднесрочное планирование является частью свопинга и представляет собой решение о добавлении процесса к множеству по крайней мере частично расположенных в основной памяти (и следовательно, доступных для выполнения) процессов

Краткосрочное планирование является решением о том, какой из готовых к выполнению процессов будет выполняться следующим

Виды планирования



Виды планирования

Важно понимать что данные три типа планирования не являются независимыми

На данной схеме продемонстрирована их вложенность



Виды планирования

Планирование оказывает большое влияние на производительность системы, поскольку именно оно определяет, какой процесс будет выполняться, а какой – ожидать выполнения

Так как для отдельных состояний процессов в рамках операционных систем существуют отдельные очереди, по сути, планирование представляет собой управление очередями с целью минимизации задержек и оптимизации производительности системы

Виды планирования



Долгосрочное планирование

Долгосрочное планирование определяет, какие программы допускаются к выполнению системой, и тем самым определяет **степень многозадачности**

Будучи допущенным к выполнению, задание (или пользовательская программа) становится процессом, который добавляется в очередь для краткосрочного планирования

В некоторых системах вновь созданный процесс добавляется к очереди среднесрочного планировщика, будучи целиком сброшенным на диск

Данный вид планировщика преимущественно используется в операционных системах **пакетной обработки**

Долгосрочное планирование

В случае использования **интерактивных программ в системах с разделением времени** запрос на запуск процесса может генерироваться действиями пользователя по подключению к системе. Пользователи не просто вносятся в очередь в ожидании, когда система обработает их запрос на подключение

Вместо этого операционная система принимает всех авторизованных пользователей до насыщения системы (пороговое значение которого определяется заранее)

После достижения состояния насыщения на все запросы на вход в систему будет получено сообщение о заполненности системы и временном прекращении доступа к ней с предложением повторить операцию входа попозже

Среднесрочное планирование

Среднесрочное планирование является частью **системы свопинга** (подробнее о том, что такое свопинг и зачем он нужен поговорим в лекции про виртуальную память)

Обычно решение о загрузке процесса в память принимается в зависимости от степени многозадачности; кроме того, в системе с отсутствием виртуальной памяти среднесрочное планирование также тесно связано с вопросами управления памятью

Таким образом, решение о загрузке процесса в память должно учитывать требования к памяти выгружаемого процесса

Краткосрочное планирование

Рассматривая частоту работы планировщика, можно сказать, что **долгосрочное** планирование выполняется сравнительно редко, принимая крупные решения о том, следует ли принять новый процесс, и какой именно

Среднесрочное планирование работает несколько чаще и принимает решения о свопинге

Краткосрочный же планировщик, известный также как **диспетчер** (dispatcher), работает чаще всего и на самом «мелкозернистом» уровне, определяя, какой именно процесс будет выполняться следующим

Краткосрочное планирование

Краткосрочный планировщик вызывается при наступлении события, которое может приостановить текущий процесс или предоставить возможность прекратить выполнение данного процесса в пользу другого. Вот некоторые примеры таких событий:

- Прерывание таймера
- Прерывания ввода-вывода
- Вызовы операционной системы
- Сигналы (например, семафоры)

Алгоритмы планирования

Теперь, когда мы узнали что такое планирование, стоит рассмотреть как именно оно происходит

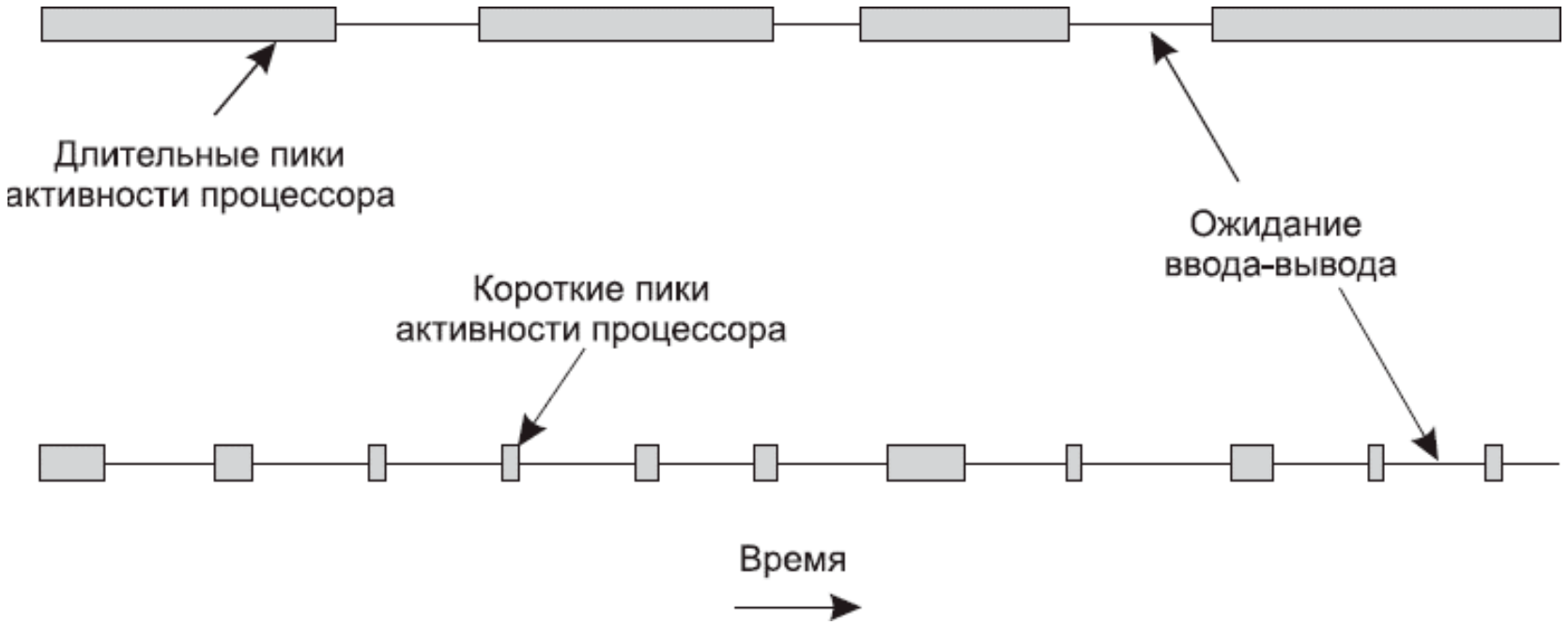
Когда идёт речь об алгоритмах планирования, то чаще всего речь именно о краткосрочном планировании

Перед тем как окунуться во все эти алгоритмы, стоит рассмотреть некоторые сложности и характеристики которые учитываются различными алгоритмами в своей работе

Первое что стоит упомянуть, различные потоки (процессы) имеют различные требования к ресурсам компьютера. Некоторые процессы, проводят основную часть своего времени за вычислениями, а другие – ожидают завершения операций ввода-вывода

Диспетчеризация и планирование

Первые процессы называются процессами, **ограниченными скоростью вычислений**, а вторые – процессами, **ограниченными скоростью работы устройств ввода-вывода**



Алгоритмы планирования

Как упоминалось немного ранее, планировщик вызывается в различных ситуациях, но почти все они так или иначе связаны с **прерываниями** (повторим уже изученное и рассмотрим кое-что новое о них в следующей лекции)

Одним из источников прерываний является аппаратный таймер. Реакция планировщика может быть на каждое такое прерывание или на каждое n-ое прерывание

По реакции на прерывания по таймеру алгоритмы планирования можно разделить на две категории: **приоритетные** и **неприоритетные**

По сути данные виды соотносятся с такими понятиями как **вытесняющее** и **невытесняющее (кооперативное) планирование**

Алгоритмы планирования

Неприоритетный алгоритм планирования выбирает запускаемый процесс, а затем просто дает ему возможность выполняться до тех пор, пока он не заблокируется (в ожидании либо завершения операции ввода-вывода, либо другого процесса), или до тех пор, пока он добровольно не освободит центральный процессор (т.е. планировщик **не вытесняет** его с процессора)

Даже если процесс будет работать в течение многих часов, он не будет приостановлен в принудительном порядке. ***В результате во время прерываний по таймеру решения приниматься не будут***

После завершения обработки прерывания по таймеру работу возобновит ранее запущенный процесс, если только какой-нибудь процесс более высокого уровня не ожидал только что истекшей задержки по времени

Алгоритмы планирования

В отличие от этого **приоритетный алгоритм планирования** предусматривает выбор процесса и предоставление ему возможности работать до истечения некоторого строго определенного периода времени (по сути до истечения некоторого **кванта времени**)

Если до окончания этого периода он все еще будет работать, планировщик приостанавливает его работу (т.е. как бы **вытесняет** его) и выбирает для запуска другой процесс (если он есть)

Осуществление приоритетного алгоритма планирования **требует наличия прерываний по таймеру**, возникающих по окончании определенного периода времени, чтобы вернуть управление центральным процессором планировщику

Если прерывания по таймеру недоступны, остается лишь использовать неприоритетное планирование

Алгоритмы планирования

Неудивительно, что в различных условиях окружающей среды требуются разные алгоритмы планирования. Это обусловлено тем, что различные сферы приложений (и разные типы операционных систем) предназначены для решения разных задач

Иными словами, предмет оптимизации для планировщика не может совпадать во всех системах. При этом стоит различать три среды:

- Пакетную
- Интерактивную
- Реального времени

Алгоритмы планирования

В **пакетных системах** не бывает пользователей, терпеливо ожидающих за своими терминалами быстрого ответа на свой короткий запрос. Поэтому для них зачастую приемлемы неприоритетные алгоритмы или приоритетные алгоритмы с длительными периодами для каждого процесса

В среде с пользователями, работающими в **интерактивном режиме**, приобретает важность приоритетность, удерживающая отдельный процесс от захвата центрального процессора, лишающего при этом доступа к службе всех других процессов

В системах, ограниченных условиями **реального времени**, как ни странно, приоритетность иногда не требуется, поскольку процессы знают, что они могут запускаться только на непродолжительные периоды времени, и зачастую выполняют свою работу довольно быстро, а затем блокируются

Задачи планирования

Чтобы создать алгоритм планирования, нужно иметь некое представление о том, с чем должен справиться толковый алгоритм. Некоторые **задачи** зависят от среды окружения, но есть и такие задачи, которые желательно выполнить в любом случае

Вот **некоторые задачи алгоритма планирования**, которых следует придерживаться при различных обстоятельствах:

- **Для любых систем**: равнодоступность, принуждение к определенной политике, баланс
- **Пакетные системы**: производительность, оборотное время, использование ЦП
- **Интерактивные системы**: время отклика, пропорциональность
- **Системы реального времени**: соблюдение предельных сроков, предсказуемость

Общие задачи планирования

Для любых систем:

Равнодоступность (беспристрастность) – предоставление каждому процессу справедливой доли времени центрального процессора

При отсутствии дополнительных указаний от пользователя или системы все процессы должны рассматриваться как равнозначные и ни один процесс не должен подвергнуться голоданию

Принуждение к определенной политике – наблюдение за выполнением установленной политики

Баланс – поддержка загруженности всех составных частей системы (стратегия планирования должна поддерживать занятость системных ресурсов)

Если центральный процессор и все устройства ввода-вывода смогут быть постоянно задействованы, то будет произведен больший объем работы в секунду, чем при простое каких-нибудь компонентов

Задачи пакетного планирования

Пакетные системы:

Производительность – это количество заданий, выполненных за один час

С учетом всех обстоятельств выполнение 50 заданий в час считается лучшим показателем, чем выполнение 40 заданий в час

Оборотное время – это среднестатистическое время от момента передачи задания на выполнение до момента завершения его выполнения

Включает время выполнения, а также время, затраченное на ожидание ресурсов, в том числе процессора

Здесь действует правило: чем меньше, тем лучше!

Алгоритмы планирования

Пакетные системы:

В качестве показателя в пакетных системах часто используется **степень задействования центрального процессора**, но это **не самый лучший показатель**

В действительности значение имеет то, сколько заданий в час выполняет система (производительность) и сколько времени занимает получение результатов задания (оборотное время).

Использование в качестве показателя степени задействованности процессора напоминает оценку машин по показателю, сколько оборотов в час делают их двигатели. В то же время информация о том, когда задействованность центрального процессора достигает 100 %, пригодится для определения момента наращивания его вычислительной мощности

Задачи интерактивного планирования

Интерактивные системы:

Наиболее важной задачей в таких системах является сведение к минимуму **времени отклика**, то есть времени между выдачей команды и получением результата

На персональных компьютерах, имеющих запущенные фоновые процессы (например, чтение из сети электронной почты и ее сохранение), пользовательский запрос на запуск программы или открытие файла должен иметь приоритет над фоновой работой

Первоочередной запуск всех интерактивных запросов будет восприниматься как хороший уровень обслуживания

Стратегия планирования должна пытаться сократить время получения ответа при максимизации количества интерактивных пользователей, время отклика для которых не выходит за заданные пределы

Задачи интерактивного планирования

Интерактивные системы:

В определенной степени к этим системам относится и задача, которую можно назвать **пропорциональностью** (**оправданием пользовательских надежд**). Пользователям свойственно прикидывать (и зачастую неверно) продолжительность тех или иных событий

Когда запрос, рассматриваемый как сложный, занимает довольно продолжительное время, пользователь воспринимает это как должное, но когда запрос, считающийся простым, также занимает немало времени, пользователь выражает недовольство

Например, если по щелчку на значке инициируется выкладывание видео объемом 500 Мбайт на облачный сервер, что занимает 60 с, пользователь, наверное, воспримет это как должное, поскольку он не ожидает, что видео будет передано за 5 с, и знает, что для этого нужно время

Задачи планирования реального времени

Системы реального времени:

В отличие от интерактивных систем системы реального времени имеют другие свойства, а значит, и другие задачи планирования

Они характеризуются наличием крайних сроков выполнения, которые обязательно или по крайней мере желательно выдерживать.

Например, если компьютер управляет устройством, которое выдает данные с определенной скоростью, отказ от запуска процесса сбора данных может привести к потере информации

Поэтому главным требованием в системах реального времени является соблюдение всех (или большей части) **крайних сроков**

Задачи планирования реального времени

Системы реального времени:

Предельный срок – при указании предельного срока завершения процесса планирование должно подчинить ему все прочие цели максимизации количества процессов, завершающихся в срок

В некоторых системах реального времени, особенно в мультимедийных системах, существенную роль играет **предсказуемость**. Редкие случаи несоблюдения крайних сроков не приводят к сбоям, но если аудиопроцесс прерывается довольно часто, то качество звука резко ухудшается

Это относится и к видеопроцессам, но человеческое ухо более чувствительно к случайным искажениям, чем глаз. Чтобы избежать подобной проблемы, планирование процессов должно быть исключительно предсказуемым и постоянным

Задачи планирования реального времени

Системы реального времени:

Предсказуемость – данное задание должно выполняться примерно за одно и то же количество времени и с одной и той же стоимостью, независимо от загрузки системы

Большие вариации времени выполнения или времени отклика дезориентируют пользователей

Это явление может сигнализировать о больших колебаниях загрузки или о необходимости дополнительной настройки системы для устранения нестабильности ее работы

Алгоритмы планирования

Давайте же рассмотрим некоторые алгоритмы планирования

Если разбить алгоритмы планирования по видам систем, то выйдет нечто подобное (здесь приведена малая часть всех алгоритмов планирования):

Планирование в пакетных системах: FCFS, SJN, SRT

Планирование в интерактивных системах: RR, Priority scheduling, SPN, MLFQ, Lottery scheduling, Fair-share scheduling, Fair share scheduling

Планирование в системах реального времени имеет свои особенности но об этом чуть позже

Алгоритмы планирования

Рассмотрим некоторые алгоритмы планирования

При объяснении планирования упоминалось что алгоритмы по реакции на таймер бывают приоритетные и неприоритетные. Стоит сделать ремарку что слово «приоритетные» в данном случае слегка отличается от того которое вы встретите далее и не стоит их путать

Как уже говорилось приоритетные и неприоритетные алгоритмы мы будем называть **вытесняющими** и **невытесняющими (кооперативными)**

Алгоритмы планирования

Невытесняющие. В этом случае находящийся в состоянии выполнения процесс продолжает выполнение до тех пор, а) пока он не завершится или б) пока не окажется в заблокированном состоянии ожидания завершения операции ввода-вывода или запроса некоторого системного сервиса

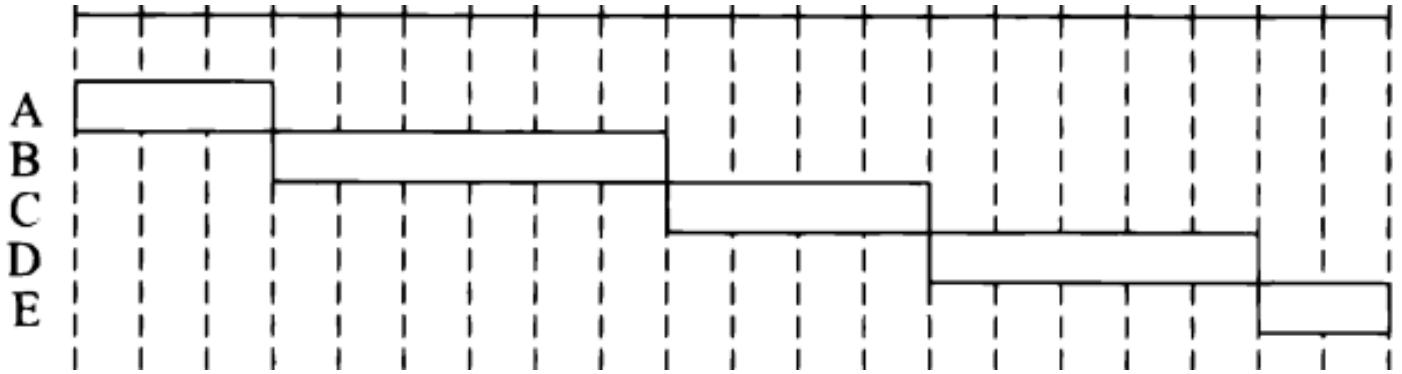
Вытесняющие. Выполняющийся в настоящий момент процесс может быть прерван и переведен операционной системой в состояние готовности к выполнению

Решение о вытеснении может приниматься при запуске нового процесса по прерыванию, которое переводит заблокированный процесс в состояние готовности к выполнению, или периодически - на основе прерываний таймера

Алгоритмы планирования FCFS

Наверное, наипростейшим из всех алгоритмов планирования будет неприоритетный алгоритм, следующий принципу **«первым пришел – первым обслужен»**

При использовании этого алгоритма центральный процессор выделяется процессам в порядке поступления их запросов. По сути, используется одна очередь процессов, находящихся в состоянии готовности



Алгоритмы планирования FCFS

Преимущества алгоритма:

- Простота реализации – легко внедрить и поддерживать
- Понятность – принцип работы интуитивно понятен
- Справедливость – все процессы получают равные шансы
- Предсказуемость – результат легко прогнозировать

Недостатки алгоритма:

- Эффект конвоя – длинные процессы блокируют выполнение коротких
- Отсутствие приоритетов – нельзя выделить важные задачи
- Высокое среднее время ожидания при наличии длинных процессов
- Низкая эффективность в интерактивных системах

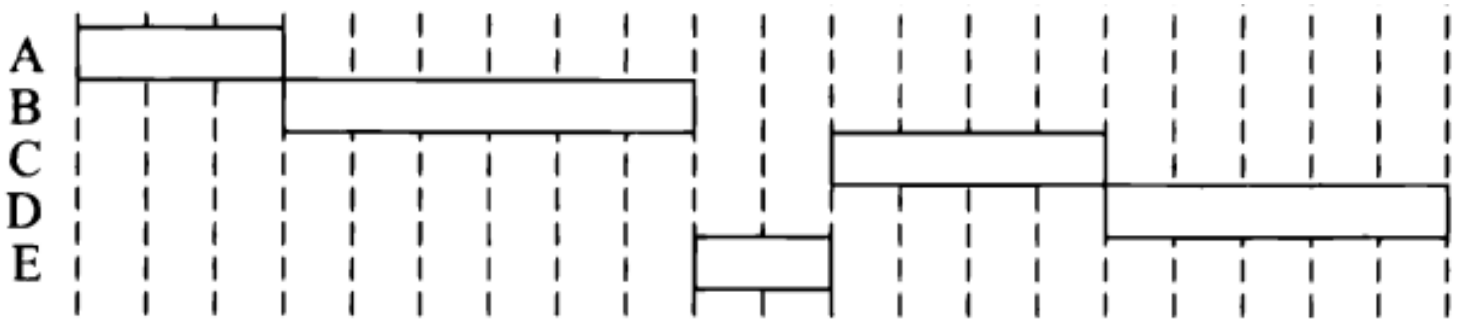
Алгоритмы планирования SJN

Теперь рассмотрим другой невытесняющий алгоритм для пакетных систем, в котором предполагается, что сроки выполнения заданий известны заранее

К примеру, в страховой компании люди могут довольно точно предсказать, сколько времени займет выполнение пакета из 1000 исковых заявлений, поскольку подобная работа выполняется ежедневно

Когда в ожидании запуска во входящей очереди находится несколько равнозначных по важности заданий, планировщик выбирает **сначала самое короткое задание**

Алгоритмы планирования SJN



Процесс	Время запуска	Время обслуживания
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Алгоритмы планирования SJN

Преимущества:

- Оптимизация производительности – минимизирует среднее время ожидания
- Эффективность – хорошо работает с короткими задачами
- Простота реализации – легко внедрить в систему
- Минимизация времени оборота процессов

Недостатки:

- Проблема голода – длинные процессы могут не получить процессор
- Необходимость предсказания времени выполнения
- Сложность оценки длительности процессов
- Риск блокировки системы короткими задачами

Алгоритмы планирования

Теперь давайте рассмотрим некоторые алгоритмы, которые могут быть использованы в интерактивных системах

Такие алгоритмы зачастую являются вытесняющими, а также основаны на такой концепции как **квантование**

Квантование означает, что каждому потоку система выделяет определенный интервал времени (**квант**), в течение которого процессор потенциально может выполнять код этого потока. По завершении выделенного кванта планировщик принудительно переключает процессор на выполнение другого готового потока (если, конечно, такой есть), переводя старый активный поток в состояние готовности. Это гарантирует, что ни один поток не захватит ЦП на непозволительно большое время

Алгоритмы планирования

Для эффективной работы ОС большое значение имеет выбор величины кванта

Некоторые планировщики умеют изменять кванты в определенных пределах, увеличивая их для тех потоков, которые не используют до конца выделенное время, например, из-за частых обращений к операциям ввода/вывода

Типичный диапазон изменения кванта – от 10 до 50 миллисекунд

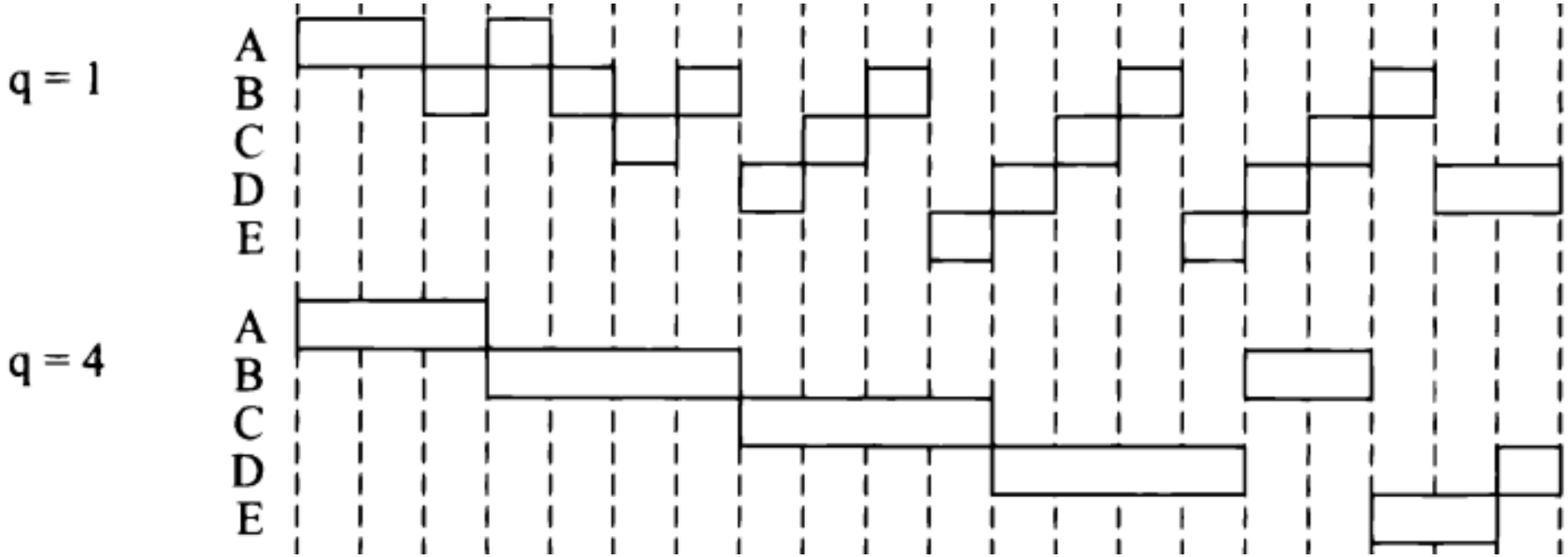
Алгоритмы планирования RR

Одним из самых старых, простых, справедливых и наиболее часто используемых считается алгоритм **циклического планирования**. Каждому процессу назначается определенный интервал времени, называемый его **квантом**, в течение которого ему предоставляется возможность выполнения

Алгоритм циклического планирования не представляет сложности в реализации: от планировщика требуется всего лишь вести список процессов, готовых к выполнению

Когда процесс **исчерпает** свой квант времени, он помещается **в конец** списка

Алгоритмы планирования RR



Алгоритмы планирования RR

Преимущества:

- Высокая отзывчивость системы
- Справедливое распределение ресурсов
- Простота реализации
- Отсутствие голодания процессов
- Эффективное использование CPU

Недостатки:

- Выбор кванта – критичный параметр
- Переключение контекста создает накладные расходы
- Не учитывает приоритеты процессов
- При слишком малых квантах – снижение производительности
- При больших квантах – ухудшение интерактивности

Алгоритмы планирования РВ

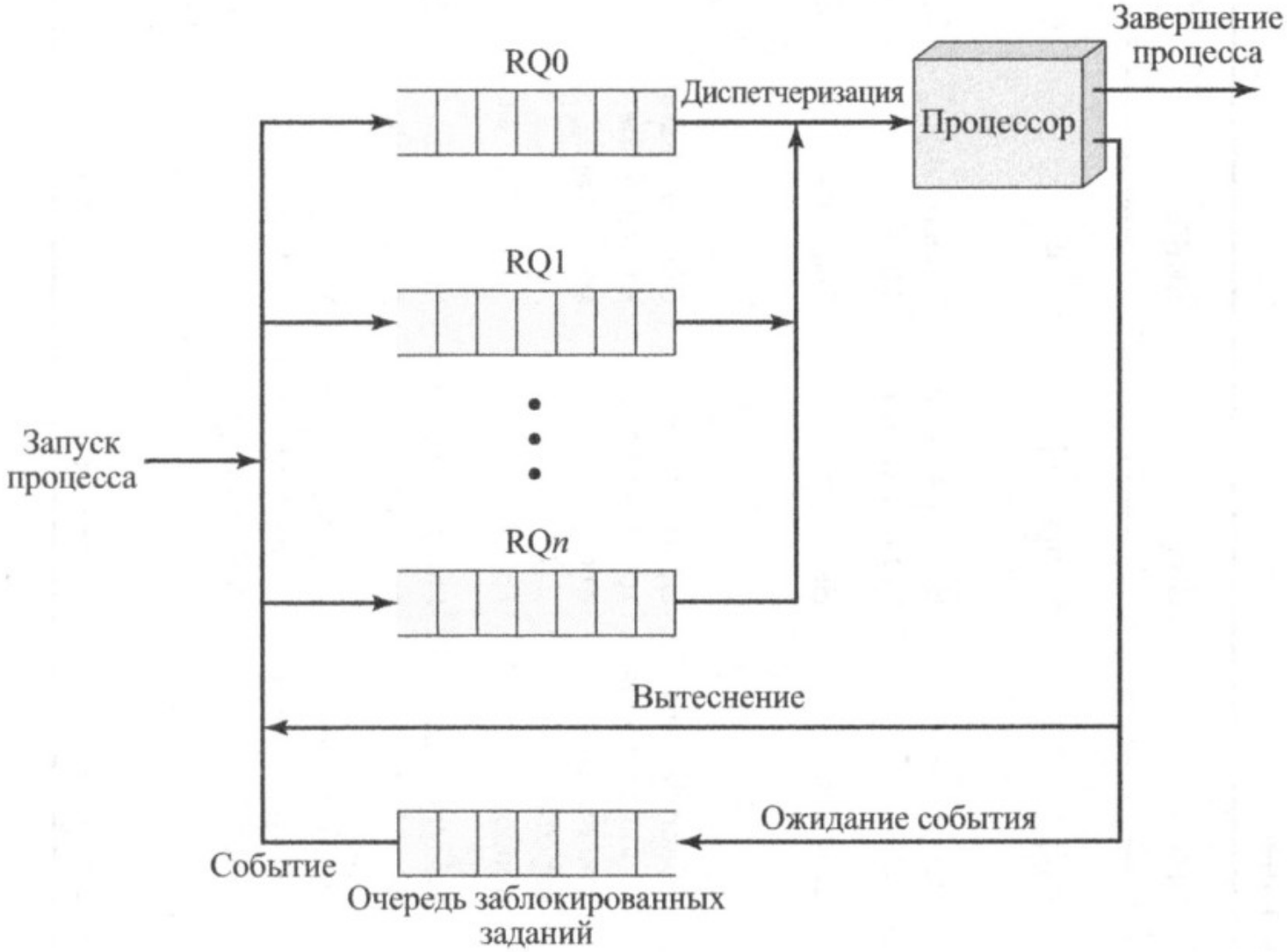
В циклическом планировании явно прослеживается предположение о равнозначности всех процессов. Зачастую люди, обладающие многопользовательскими компьютерами и работающие на них, имеют на этот счет совершенно иное мнение

К примеру, в университете иерархия приоритетности должна нисходить от декана к факультетам, затем к профессорам, секретарям, техническим работникам, а уже потом к студентам

Необходимость учета внешних факторов приводит к **приоритетному планированию**

Основная идея проста: каждому процессу присваивается значение приоритетности и запускается тот процесс, который находится в состоянии готовности и имеет наивысший **приоритет**

Алгоритмы планирования РВ



Алгоритмы планирования РВ

Приоритет – некоторое значение которое определяет важность процесса и влияет на частоту запуска потока и, возможно, на величину выделяемого кванта

Процессы могут иметь разную степень важности: системные – более высокую (иначе ОС не сможет решать свои задачи), прикладные – менее высокую

Многие ОС позволяют группировать потоки по их важности, выделяя три группы, или класса:

- Потоки реального времени с максимально высоким уровнем приоритета
- Системные потоки с меньшим уровнем приоритета
- Прикладные потоки с самым низким приоритетом

Алгоритмы планирования РВ

Внутри каждой группы выделяется свой диапазон возможных значений приоритетов, причем эти диапазоны между собой не пересекаются, т.е. максимально возможный приоритет прикладного потока всегда будет строго меньше минимально возможного приоритета для системных потоков

Внутри каждой группы могут использоваться разные алгоритмы управления приоритетами

Алгоритмы планирования РВ

Одна из основных проблем в такой чисто приоритетной схеме планирования состоит в том, что процессы с низким приоритетом могут оказаться в состоянии **голодания**

Это будет происходить при постоянном поступлении новых готовых к выполнению процессов с высоким приоритетом

Если такое поведение нежелательно, приоритет процесса может изменяться с его «возрастом» или историей выполнения

Соответственно приоритеты бывают – **динамическими** и **статическими**

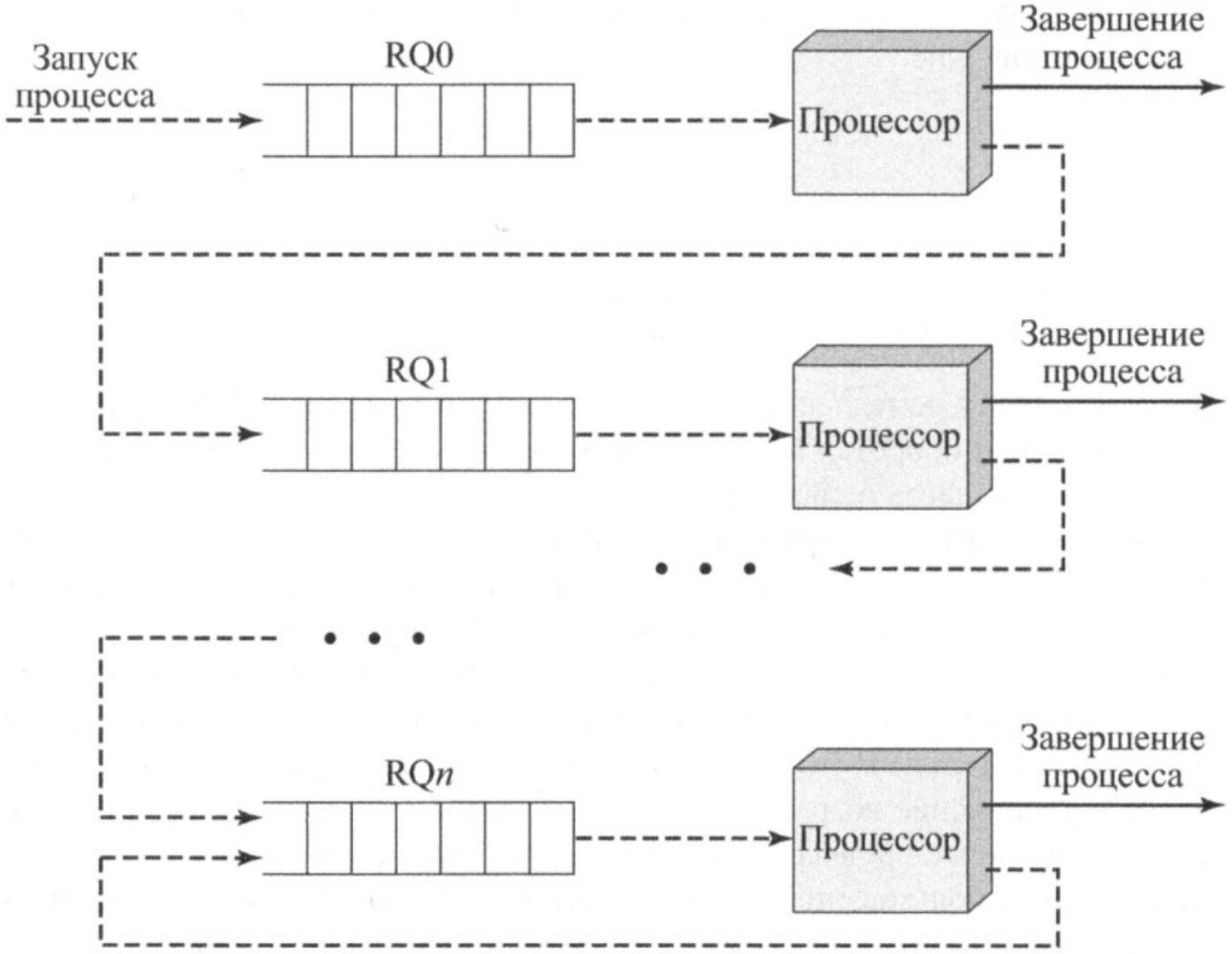
Алгоритмы планирования РВ

Если приоритет потока может меняться системой, то такие приоритеты называют **динамическими**, иначе – **статическими**. Конечно, реализация фиксированных приоритетов гораздо проще, тогда как динамические приоритеты позволяют реализовать более справедливое распределение процессорного времени.

Потоки, интенсивно использующие внешние устройства, очень часто блокируются до завершения выделенного кванта времени, т.е. не используют эти кванты полностью, и их приоритет следует **повысить** для быстрой активации, что обеспечивает большую загрузку относительно медленных внешних устройств

Если поток полностью расходует выделенный квант, система может после его приостановки **уменьшить** приоритет. Этим достигается более равномерная загрузка вычислительной системы в целом

Алгоритмы планирования РВ



Алгоритмы планирования РВ

Довольно интересной и часто используемой разновидностью приоритетов являются так называемые **абсолютные приоритеты**: как только среди готовых процессов появляется поток, приоритет которого выше, чем приоритет текущего активного процесса, этот активный поток досрочно прерывается с передачей процессора более приоритетному процессу

Данный вид приоритетов показывает более высокую эффективность в системах реального времени

Алгоритмы планирования

Говоря о приоритетах стоит сразу отметить как это устроено в операционных системах Windows и Linux

Если говорить о Windows, то стоит вспомнить что единицей планирования является – поток

Вместе с тем приоритет в Windows имеется не только у потоков, но и у процессов

В Linux как мы помним система не различает понятия потока и процесса, поэтому приоритет не разделяется также как в Windows

Алгоритмы планирования Windows

Потоки планируются на основе их приоритета планирования

Каждому потоку назначается приоритет планирования. Уровни приоритета варьируются от нуля (самый низкий приоритет) до 31 (наивысший приоритет)

Только поток обнуления страниц памяти может иметь нулевой приоритет

Поток обнуления страниц – это системный поток, отвечающий за обнуление всех свободных страниц, если нет других потоков, которые должны выполняться

Система рассматривает все потоки с одинаковым приоритетом как равные

Алгоритмы планирования Windows

Система назначает кванты времени циклическим перебором всем потокам с наивысшим приоритетом. Если ни один из этих потоков не готов к запуску, система назначает кванты времени циклическим перебором всем потокам со следующим наивысшим приоритетом

Если поток с более высоким приоритетом становится доступным для выполнения, система перестает выполнять поток с более низким приоритетом (не позволяя ей завершить использование кванта времени) и назначает полный квант времени потоку с более высоким приоритетом

Алгоритмы планирования Windows

Приоритет каждого потока определяется следующими критериями:

- Класс приоритета процесса
- Уровень приоритета потока в классе приоритета процесса

Класс и уровень приоритета объединяются для формирования **базового приоритета потока**

Именно базовый приоритет потока может принимать значения от 1 до 31

Алгоритмы планирования Windows

Каждый процесс принадлежит к одному из следующих классов приоритетов:

- IDLE_PRIORITY_CLASS
- BELOW_NORMAL_PRIORITY_CLASS
- NORMAL_PRIORITY_CLASS
- ABOVE_NORMAL_PRIORITY_CLASS
- HIGH_PRIORITY_CLASS
- REALTIME_PRIORITY_CLASS

По умолчанию класс приоритета процесса – NORMAL_PRIORITY_CLASS

Алгоритмы планирования Windows

Используйте **CreateProcess**, чтобы указать класс приоритета дочернего процесса при его создании. Если вызывающий процесс IDLE_PRIORITY_CLASS или BELOW_NORMAL_PRIORITY_CLASS, новый процесс наследует этот класс

Используйте функцию **GetPriorityClass** для определения текущего класса приоритета процесса и функцию **SetPriorityClass**, чтобы изменить класс приоритета процесса

Алгоритмы планирования

Ниже приведены уровни приоритета в каждом классе приоритета:

- THREAD_PRIORITY_IDLE
- THREAD_PRIORITY_LOWEST
- THREAD_PRIORITY_BELOW_NORMAL
- THREAD_PRIORITY_NORMAL
- THREAD_PRIORITY_ABOVE_NORMAL
- THREAD_PRIORITY_HIGHEST
- THREAD_PRIORITY_TIME_CRITICAL

Все потоки создаются с помощью THREAD_PRIORITY_NORMAL. Это означает, что приоритет потока совпадает с классом приоритета процесса

Алгоритмы планирования Windows

В отличие от процессов приоритет потока не может быть задан при создании потока

Используйте функцию **GetThreadPriority** для определения текущего приоритета потока и функцию **SetThreadPriority**, чтобы изменить приоритет потока

Типичная стратегия заключается в использовании THREAD_PRIORITY_ABOVE_NORMAL или THREAD_PRIORITY_HIGHEST для потока ввода процесса, чтобы обеспечить реагирование приложения на запросы пользователя

Алгоритмы планирования Windows

Фоновые потоки, особенно те, которые интенсивно используют процессор, можно задать для THREAD_PRIORITY_BELOW_NORMAL или THREAD_PRIORITY_LOWEST, чтобы обеспечить их вытеснять при необходимости

Однако если у вас есть поток, ожидающий другого потока с более низким приоритетом для выполнения какой-то задачи, обязательно заблокируйте выполнение ожидающего потока с высоким приоритетом. Для этого используйте функцию ожидания, критический раздел или функцию **Sleep**, **SleepEx** или **SwitchToThread**

Алгоритмы планирования Windows

		Process Classes					
		Realtime	High	Above Normal	Normal	Below Normal	Idle
Thread Priorities	Time Critical	31	15	15	15	15	15
	Highest	26	15	12	10	8	6
	Above Normal	25	14	11	9	7	5
	Normal	24	13	10	8	6	4
	Below Normal	23	12	9	7	5	3
	Lowest	22	11	8	6	4	2
	Idle	16	1	1	1	1	1

Алгоритмы планирования Windows

Как уже говорилось для более эффективной работы приоритетного планирования стоит использовать динамические приоритеты, поэтому в Windows приоритеты потоков могут динамически меняться

Каждый поток имеет динамический приоритет. Это приоритет, который использует планировщик, чтобы определить, какой поток следует выполнить

Изначально динамический приоритет потока совпадает с базовым приоритетом

Система может повысить и понизить динамический приоритет, чтобы гарантировать, что она реагирует и что потоки не голодают в течение времени процессора

Система не повышает приоритет потоков с базовым уровнем приоритета от 16 до 31

Динамические повышения приоритета получают только потоки с базовым приоритетом от 0 до 15

Алгоритмы планирования Windows

Функцию повышения приоритета можно отключить, вызвав функцию **SetProcessPriorityBoost** или **SetThreadPriorityBoost**

Чтобы определить, отключена ли эта функция, вызовите функцию **GetProcessPriorityBoost** или **GetThreadPriorityBoost**

После повышения динамического приоритета потока планировщик уменьшает этот приоритет на один уровень каждый раз, когда поток завершает квант времени, пока поток не вернется к базовому приоритету

Динамический приоритет потока никогда не меньше базового приоритета

Алгоритмы планирования Linux

Приоритет процесса в Linux означает, насколько больше процессорного времени будет отдано этому процессу по сравнению с другими

Существует два вида приоритетов, связанных с каждым процессом:

- Значение «любезности» (nice), которое варьируется от -20 (самый высокий приоритет) до 19 (самый низкий приоритет)
- Приоритет реального времени, варьирующийся от 0 до 99

Когда мы хотим установить приоритет процесса, мы изменяем значение «любезности» процесса. Причем, уменьшать приоритет можно с правами обычного пользователя, но чтобы его увеличить, нужны права суперпользователя

Алгоритмы планирования Linux

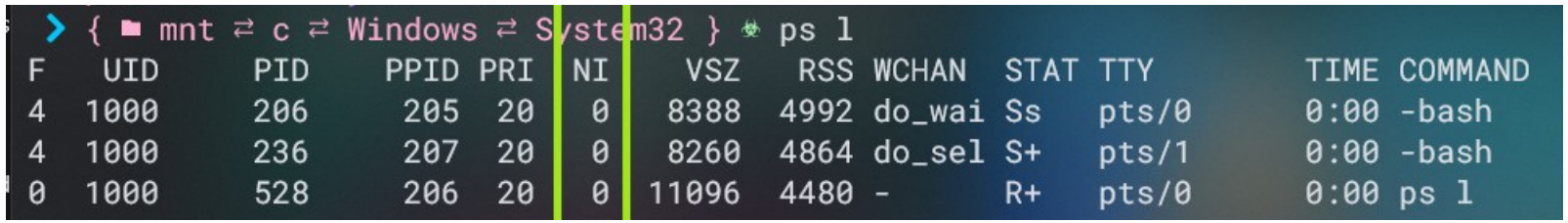
С помощью команды nice вы можете указать приоритет для запускаемого процесса:

nice -n 10 apt-get upgrade

С помощью команды renice изменить приоритет для уже существующего по его pid (-p) или по его имени (-u):

renice -n 10 -p 1224

Узнать значение любезности можно через ту же утилиту ps:



Алгоритмы планирования Linux

Приоритет nice и приоритет планировщика процессов ядра ОС – разные числа

Число nice – приоритет, который пользователь хотел бы назначить процессу

Приоритет планировщика – действительный приоритет, назначенный процессу планировщиком

Планировщик может стремиться назначить процессу приоритет, близкий к nice, но это не всегда возможно, так как в системе может выполняться множество процессов с разными приоритетами

Алгоритмы планирования Linux

Приоритет nice является атрибутом процесса и, как и другие атрибуты, наследуется дочерними процессами. В выводе утилит top, ps, htop и др. приоритет nice называется «NI» – сокращение от «nice», а приоритет планировщика – «PRI» – сокращение от «priority»

Обычно, NI = PRI - 20, но это верно не всегда. По умолчанию NI=0, соответственно PRI=20

Планировщик процессов ядра ОС Linux поддерживает приоритеты от 0 (реальное время) до 139 включительно

Приоритеты -20...+19 утилиты или команды nice соответствуют приоритетам 100...139 планировщ



Алгоритмы планирования Linux

Стандарт POSIX также определяет следующие функции:

- **nice**
- **renice**
- **setpriority**
- **getpriority**

Хотя функции nice и setpriority явно отличаются в названии и можно было бы подумать что они работают с разными приоритетами, но нет – разница данных функций по сути лишь в том, чей приоритет они могут менять

Алгоритмы планирования

Ещё одним важным моментом планирования потоков/процессов в современных системах является многопроцессорность таковых

В однопроцессорной системе планирование ведется в одном измерении. Здесь нужно многократно отвечать лишь на один вопрос: какой поток должен быть запущен следующим?

В мультипроцессорных системах планирование ведется в **двух измерениях**. Планировщик должен решить, **какой поток** запускать и на **каком центральном процессоре** следует это сделать. Это дополнительное измерение существенно усложняет планирование на мультипроцессорах

Алгоритмы планирования

Другим усложняющим фактором является то, что в некоторых системах все потоки не связаны друг с другом в силу принадлежности к разным процессам и не могут ничего сделать друг с другом

В других системах они сведены в группы, где все они принадлежат одному и тому же приложению и работают вместе

Остановимся лишь на первом варианте, он является более простым случаем планирования

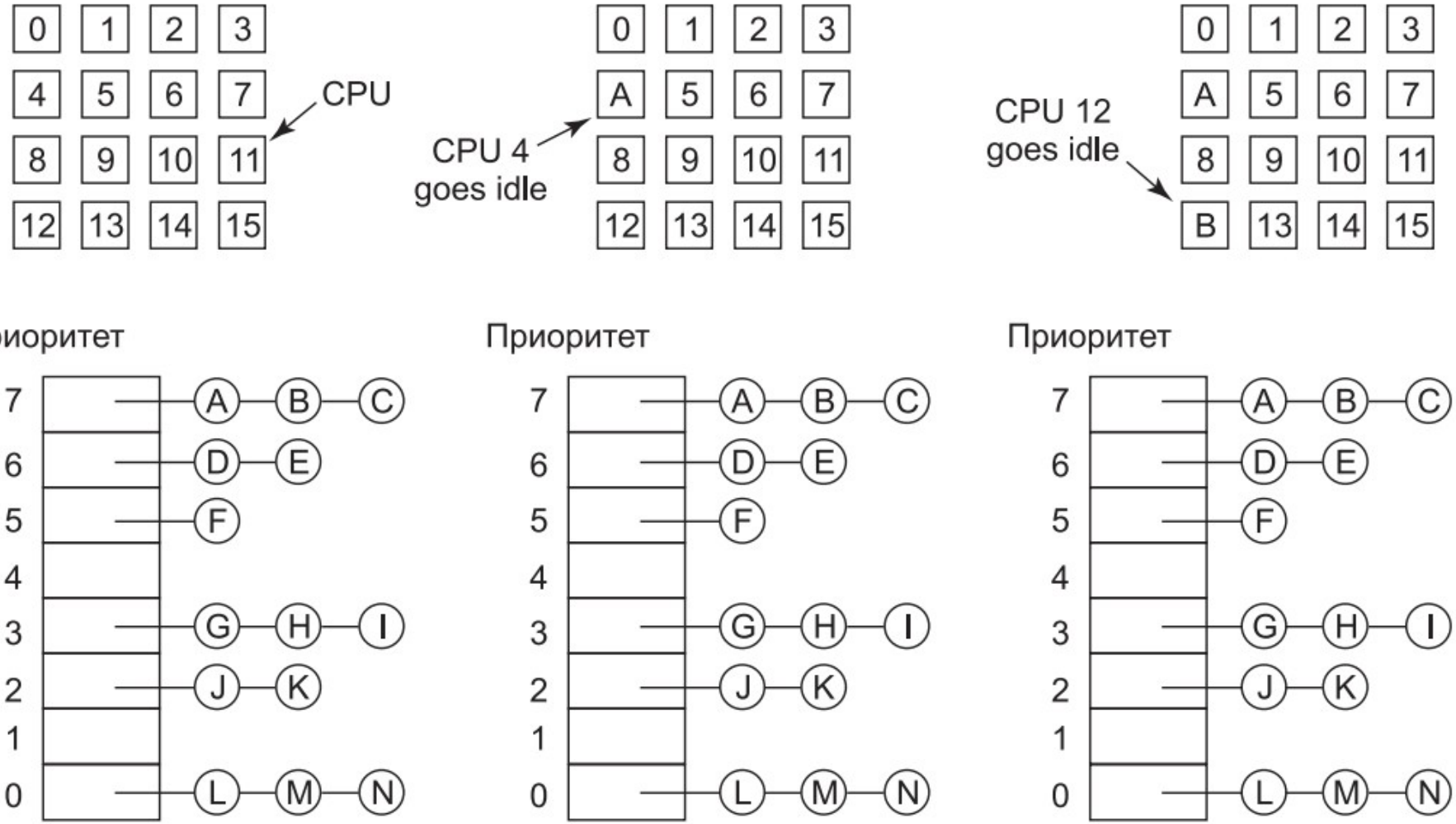
Алгоритмы планирования

Простейший алгоритм планирования для работы с независимыми потоками заключается в поддержке для готовых к работе потоков единой структуры данных для всей системы, возможно, в виде простого списка, но, скорее всего, в виде набора списков для потоков с разными приоритетами

Наличие единой структуры данных, используемой всеми центральными процессорами, позволяет этим процессорам работать в режиме разделения времени, во многом напоминающем такой режим на однопроцессорной системе

За счет этого также обеспечивается автоматическая сбалансированность нагрузки благодаря исключению случаев, когда один центральный процессор простаивает, в то время как другие перегружены

Алгоритмы планирования



Алгоритмы планирования

У этого подхода имеется два недостатка:

- Потенциальная конкуренция за доступ к структуре данных, используемой при планировании по мере роста количества центральных процессоров
- Обычные издержки при переключении контекста, когда поток блокируется в ожидании завершения операций ввода-вывода

Алгоритмы планирования

Еще один вопрос, играющий роль при планировании, связан с тем фактом, что пока все центральные процессоры находятся в равноправном положении, у некоторых центральных процессоров все же имеются некие привилегии

В частности, когда поток A отработал длительное время на центральном процессоре k, то кэш-память этого процессора будет заполнена блоками, необходимыми потоку A. Если A вскоре опять получит возможность выполнения, то эффективнее всего он будет работать на центральном процессоре k, поскольку в его кэше все еще могут находиться нужные потоку A блоки

Наличие заранее загруженных блоков повысит число реализаций запросов за счет кэша, а следовательно, и скорость работы потока

Алгоритмы планирования

На некоторых мультипроцессорах все это берется в расчет и используется так называемое **родственное планирование** (**affinity scheduling**)

Основной замысел состоит в стремлении выполнять поток на том же самом центральном процессоре, на котором он запускался в последний раз

Один из способов поддержания такой родственной связи заключается в использовании **двухуровневого алгоритма планирования** (**two-level scheduling algorithm**)

При создании поток назначается конкретному центральному процессору, к примеру тому, у которого в данный момент наименьшая нагрузка. Это назначение потока центральному процессору является **верхним уровнем алгоритма**

Алгоритмы планирования

В результате такой политики каждый центральный процессор располагает своим собственным набором потоков

А непосредственное планирование потоков представляет собой **нижний уровень алгоритма**. Он исполняется каждым центральным процессором по отдельности с использованием приоритетов или каких-нибудь других средств выбора

Родственность максимально поддерживается за счет стремления выполнять поток в течение всего цикла его существования на одном и том же центральном процессоре. Но если у центрального процессора отсутствуют потоки для запуска, он не простаивает, а забирает поток у другого процессора

Алгоритмы планирования

У двухуровневого планирования есть три преимущества:

Во-первых, оно распределяет нагрузку среди имеющихся центральных процессоров примерно поровну

Во-вторых, по возможности используется родственность содержимого кэша запускаемому потоку

В-третьих, предоставление каждому центральному процессору своего собственного списка готовых потоков сводит к минимуму конкуренцию за использование списков готовности, поскольку попытки воспользоваться списком, принадлежащим другому центральному процессору, предпринимаются довольно редко

Алгоритмы планирования

Как Windows, так и Linux предоставляют инструментарий для работы с родственностью между потоками/процессами и процессорами

В рамках каждого процесса и потока существует такое понятие как **«маска родственности» (affinity mask)**

Маска родственности – это битовая маска указывающая планировщику операционной системы на каких процессорах должен выполняться процесс или поток

Например, если вы имеете 6 физических и 12 логических ядер, то маска по умолчанию будет иметь вид 0000111111111111

Где значение бита с номером n обозначает допустимость выполнения процесса/потока на процессоре с номером n

Алгоритмы планирования Windows

В Windows для получения значения данной маски используется функция **GetProcessAffinityMask** и функция **SetProcessAffinityMask** для её изменения

Для установки маски отдельного потока используется **SetThreadAffinityMask**

Также стоит отметить что с помощью **GetProcessAffinityMask** можно получить значение системной маски (она применяется ко всем процессам по умолчанию)

Также стоит отметить, что маска процесса является подмножеством системной маски, а маска потока – маски процесса (т.е. если в маске для процессора с номером 0 установлено значение 0, то маска процесса также будет содержать там 0)

Алгоритмы планирования Windows

В дополнение к этому Windows API предоставляет возможность установки предпочтительного процессора для потока: **SetThreadIdealProcessor**

Когда вы указываете предпочтительный процессор для потока, планировщик запускает этот поток на указанном процессоре, если это возможно

Важно понимать, что:

- Это не гарантирует, что именно указанный процессор будет выбран
- Данная функция лишь предоставляет планировщику полезную подсказку
- Система будет стараться следовать указанной рекомендации, но окончательное решение принимает планировщик с учетом текущей нагрузки и других факторов

Алгоритмы планирования Linux

В Linux для получения значения маски родственности используется функция **sched_getaffinity** и функция **sched_setaffinity** для её изменения

Для получения\установки маски отдельного POSIX-потока необходимо использовать **pthread_getaffinity_np\pthread_setaffinity_np**

В отличие от Windows в Linux нельзя получить системную маску, однако в остальном принцип работы масок очень похож

Для работы с масками в Linux также присутствует набор функций **CPU_***

Алгоритмы планирования

В системах реального времени, в которых главным критерием эффективности является обеспечение временных характеристик вычислительного процесса, планирование имеет особое значение

Любая система реальною времени должна реагировать на сигналы управляемого объекта в течение заданных временных ограничений

Необходимость тщательного планирования работ облегчается тем, что **в системах реального времени весь набор выполняемых задач известен заранее**

Кроме того, часто в системе **имеется информация о** временах выполнения задач, моментах активизации, предельных допустимых сроках ожидания ответа и т. д.

Алгоритмы планирования

При разработке алгоритмов планирования для систем реального времени необходимо учитывать, какие последствия в этих системах возникают при несоблюдении временных ограничений

Если эти последствия катастрофичны, как, например, для системы управления полетами или атомной электростанцией, то операционная система реального времени, на основе которой строится управление объектом, называется **жёсткой**

Если же последствия нарушения временных ограничений не столь серьезны, то есть сравнимы с той пользой, которую приносит система управления объектом, то система является **мягкой**. Примером мягкой системы реального времени является система резервирования билетов

Алгоритмы планирования

В жестких системах реального времени время завершения выполнения каждой из критических задач должно быть гарантировано для всех возможных сценариев работы системы

При построении расписания надо иметь в виду, что для некоторых наборов задач в принципе невозможно найти расписания, в котором бы удовлетворялись заданные временные характеристики. С целью определения возможности существования расписания могут быть использованы различные критерии

Точные критерии, гарантирующие наличие расписания, являются очень сложными в вычислительном отношении

Алгоритмы планирования

В мягких системах реального времени предполагается, что заданные временные ограничения могут иногда нарушаться, поэтому здесь обычно применяются менее затратные способы планирования

В зависимости от характера возникновения запросов на выполнение задач полезно разделять их на два типа: **периодические** и **спорадические**

Начиная с момента первоначального запроса, все будущие моменты запроса периодической задачи можно определить заранее путем прибавления к моменту начального запроса величины, кратной известному периоду

Время поступления запросов на выполнение спорадических задач заранее не известно

Алгоритмы планирования

Алгоритмы планирования работы систем реального времени могут быть **статическими** или **динамическими**. Первый из них предусматривает принятие решений по планированию еще до запуска системы, а второй – их принятие в реальном времени, после того как начнется выполнение программы

Статическое планирование работает только при условии предварительного обладания достоверной информацией о выполняемой работе и о крайних сроках, которые нужно соблюсти

Алгоритмы динамического планирования подобных ограничений не имеют

Моменты перепланировки

Для реализации алгоритма планирования ОС должна получать управление всякий раз, когда в системе происходит событие, требующее перераспределения процессорного времени

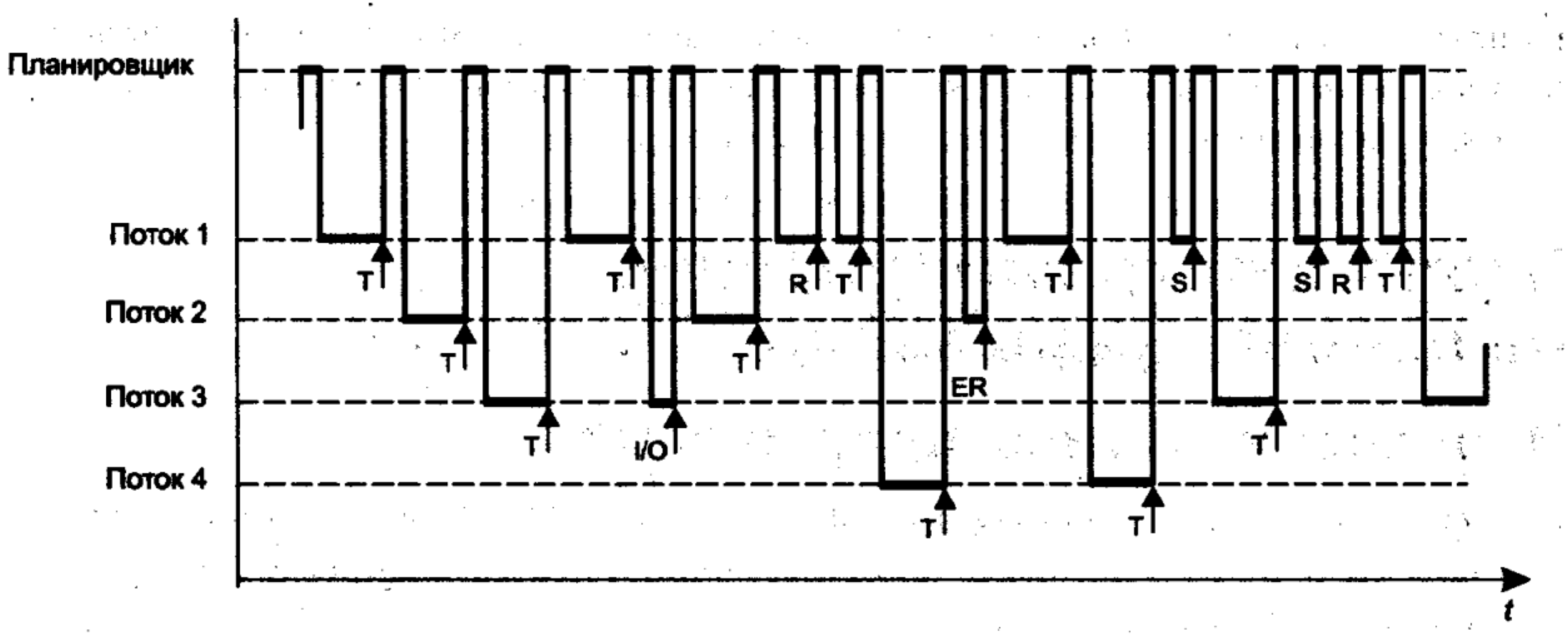
К событиям, приводящим к необходимости перепланирования процессов, могут быть отнесены следующие:

- (T) Прерывание от таймера, сигнализирующее, что ***время, отведенное активной задаче на выполнение, закончилось***. Планировщик переводит задачу в состояние готовности и выполняет перепланирование
- (I/O) Активная задача выполнила ***системный вызов, связанный с запросом на ввод-вывод или на доступ к ресурсу,*** который в настоящий момент занят (например, файл данных). Планировщик переводит задачу в состояние ожидания и выполняет перепланирование

Моменты перепланировки

- (R) Активная задача выполнила ***системный вызов, связанный с освобождением ресурса***. Планировщик проверяет, не ожидает ли этот ресурс какая-либо задача. Если – да, то эта задача переводится из состояния ожидания в состояние готовности. При этом, возможно, что задача, которая получает ресурс, имеет более высокий приоритет, чем текущая активная задача. После перепланирования более приоритетная задача получает доступ к процессору, вытесняя текущую задачу
- (S) Внешнее (аппаратное) прерывание, которое сигнализирует о ***завершении периферийным устройством операции ввода-вывода***, переводит соответствующую задачу в очередь готовых, и выполняется перепланирование
- (ER) Внутреннее прерывание сигнализирует ***об ошибке***, которая произошла в результате выполнения активной задачи. Планировщик снимает задачу и выполняет перепланирование

Моменты перепланировки



Операционные системы

Лекция 6

Диспетчеризация и планирование